

L7: Introduction to the CSE Machine

CS1101S: Programming Methodology

Martin Henz

October 6, 2023

Outline

- Need for State; Limitations of Substitution Model
- Introducing State
- Expressions and Declarations
- Control Statements
- Mutable Data

Announcements

- **Robotics Missions (and Quest)**

- Mission: Robotic Trials
 - Opened on Wednesday; to complete and be graded in Studio S8
- Mission: Moving about on Planet Y
- Mission: Finding ELDRIC
- Quest: PID Stop



- **Sumobot Competition / Festival**

- 21-Oct-2023, Saturday, 10am-2pm, in COM3-MPH, lunch provided
- Two competitions: **Sumobot & Robot Talent Show**
- Studios have to sign up next week
- Rules will be announced next week

Announcements

- **Forum Hours** in Week 8, as usual: 6.30pm to 8:30pm

Announcements

- **Source Academy: Get involved!**
- CP3108 is offered exclusively to you, the current CS1101S students in Sem 2 this Academic Year.
- Choose between CP3108A (2 Units) and CP3108B (4 Units)
- Pair-programming!
- We will publish “job descriptions” and accept applications until Week 13.
- Look out for our announcements.

Announcements

**Sailing voyage
coming up!**

To see what to expect:

**[https://www.youtube.com/
@nusseafarers4547](https://www.youtube.com/@nusseafarers4547)**



The graphic features a background image of a sailboat on the water. Overlaid on this are three circular inset photos: the left one shows two people on a boat, the middle one shows a sunset over the ocean, and the right one shows people in a yellow kayak. A dashed orange line with an arrow points from the text 'More details in the sign up sheet' to the QR code.

SET SAIL WITH SOC

Sailing the Java Sea

\$S2800
/Person

CCSGP & THE NUS SEAFARERS
WELCOME SOC STUDENTS TO SET
SAIL ON THE FIRST EVER SOC
SAILING TRIP, TRAVELING FROM
BANGKA TO JAKARTA

STUDENT SUBSIDY:
2000 SGD; APPLY FOR
ELIGIBILITY CHECK.

More details
in the sign up
sheet

Date: 27 Dec 2023 - 7 Jan 2024
Eligibility: Full-time SoC
undergraduate students



Collaboration on Missions/Quests

- You **can discuss** missions/quests with friends and Avengers
- You **must** do the **actual programming yourself**
 - **Do not copy** programs from other students or previous batches
- We focus on **individual learning** and **skills**
 - We want to know your **progress**, and help you to overcome your weaknesses
- Submitting other people's work as your own is considered **Plagiarism**, and is an **offence** and will be **punished** by NUS
- Exception: Collaborate with Studio team on Robot Missions/Quests
- **NUS policy on plagiarism**
 - <https://www.comp.nus.edu.sg/cug/plagiarism/>
 - <https://www.usp.nus.edu.sg/curriculum/plagiarism/>

Module Overview

- **Unit 1 — Functions** (textbook Chapter 1)
 - Getting acquainted with the elements of programming, using **functional abstraction**
 - Learning to read programs, and using the **substitution model**
 - Example applications: runes, curves
- **Unit 2 — Data** (textbook Chapter 2)
 - Getting familiar with data: pairs, lists, trees
 - Searching in lists and trees, sorting of lists
 - Example application: sound processing

Module Overview

- **Unit 3 — State** (parts of textbook Chapter 3)
 - Programming with **stateful abstractions**
 - **Mutable data** processing
 - Introduction to **CSE machine**
 - Environments, blocks, functions in the **CSE machine**
 - Arrays, loops, searching in and sorting of arrays
 - Example applications: robotics, image/video processing
- **Unit 4 — Beyond** (parts of textbook Chapters 3 and 4)
 - Streams
 - Metalinguistic abstraction
 - Computing with Register Machines / Logic Programming / Concurrency

} today

Source §3

- **New language: Source §3**
 - See language documentation at https://docs.sourceacademy.org/source_3/
 - Source Academy [Playground](#) is using **Source §3** from now on
 - Example applications of Unit 3: Robotics and video processing

Outline

- Need for State; Limitations of Substitution Model
- Introducing State
- Expressions and Declarations
- Control Statements
- Mutable Data

So far ...

- Our computation has been ***functional***
 - Given same argument, function always returns same result*
 - ***Memoryless*** / ***stateless*** — each function call is independent of the past, and does not affect the future
 - Makes it easy to reason about program behavior
 - Use ***substitution model of evaluation***

Functional Programming

- **Example:**
 - `factorial(5)` always gives 120
 - No matter how many times you call it, or when you call it
- Compare with a **bank account**:
 - Suppose balance starts with 100.

`withdraw(40);` → 60

`withdraw(40);` → 20

`withdraw(40);` → 20 "Insufficient funds"

`withdraw(15);` → 5

State

- Identical calls to `withdraw` produce different results
- Bank account has “**memory**”
 - It remembers something about the past
 - It has ***state***
- Functional programming does not allow our programs to have state
 - We need to introduce state with ***assignment***

State in robots—Using Variables and Assignment

```
let distance = 100;           // let introduces variable

distance = 80;                // assignment changes variable

distance = distance - 30;     // variable access in assignment

// btw

const distance = 100;

distance = 80;
// error: assignment not allowed
```

Data structures should change, too

```
const speed = pair(2.5, 4.0); // vector in 2D  
// speed is [2.5, 4.0]
```

```
set_head(speed, 5.0);          // change head  
// speed is now [5.0, 4.0]
```

```
// btw
```

```
speed = pair(9.0, 7.0);  
// error: assignment not allowed
```

Iterative Processes are Awkward

```
function factorial(n) {  
    function helper(i, result) { // awkward  
        return i <= n  
            ? helper(i + 1, result * i)  
            : result;  
    }  
    return helper(1, 1);  
}  
factorial(5);
```

[Show in
Playground](#)

Stateless Iterative Processes are Awkward

```
function factorial(n) {  
  function helper(i, result) {    // awkward  
    return i <= n  
      ? helper(i + 1, result * i)  
      : result;  
  }  
  return helper(1, 1);  
}
```

```
function factorial(n) {    // more readable Show in Playground  
  let i = 1;  
  let result = 1;  
  while (i <= n) {  
    result = result * i;  
    i = i + 1;  
  }  
  return result;  
}
```

Variable Declaration Statement

```
let name = expression;
```

- Declares a **variable** *name* in the current scope and initializes its value to the value of *expression*
- From now on, *name* will evaluate to the value of *expression*

Assignment Statement

name = *expression*;

- *name* is a **variable**; not evaluated
- *expression* is evaluated, then its value is **assigned** to the variable *name*
- From now on, *name* will evaluate to the value of *expression*

Example

```
let distance = 100;
```

```
distance; → 100
```

```
distance = 80;
```

```
distance; → 80
```

```
distance = distance - 30;
```

```
distance; → 50
```

Substitution Model Breaks Down

- Consider

```
let distance = 100;  
distance = 80;  
distance = distance - 30;
```

- Using **substitution model** we get:

```
100 = 80;           // nonsense
```

```
distance = 100 - 30; // wrong
```

Substitution Model Breaks Down

- Consider

```
let balance = 100; // replace name balance by 100
function f() {
    return balance; // should we replace? when?
}
f(); // should return 100
balance = 80;
f(); // should return 80
```

- We need to revise our model!

Substitution Model Breaks Down

- Consider

```
let distance = 100;  
distance = 80;  
distance = distance - 30;
```

- Using **substitution model** we get:

```
100 = 80;           // nonsense
```

```
distance = 100 - 30; // wrong
```

Why Substitution Model Breaks Down?

- Substitution model considers a constant/variable as **just a name for a value**
 - Its value will not change
 - Therefore, one can be substituted for the other
- But **assignment** considers a variable as a “**container**” **holding a value**
 - The contents of the container may be **changed over time**
 - The container is maintained in a structure called an ***environment***

Assignment: Pros

- Assignment allows us to create objects with ***state***
- State allows objects to behave differently over time
- Iterative processes can be described more naturally

Assignment: Cons

- Harder to reason about programs
 - Harder to debug
 - Harder to verify correctness
- **Substitution model of evaluation breaks down!**
 - Not powerful enough to explain state
 - Need a more sophisticated model — ***CSE Machine***

Outline

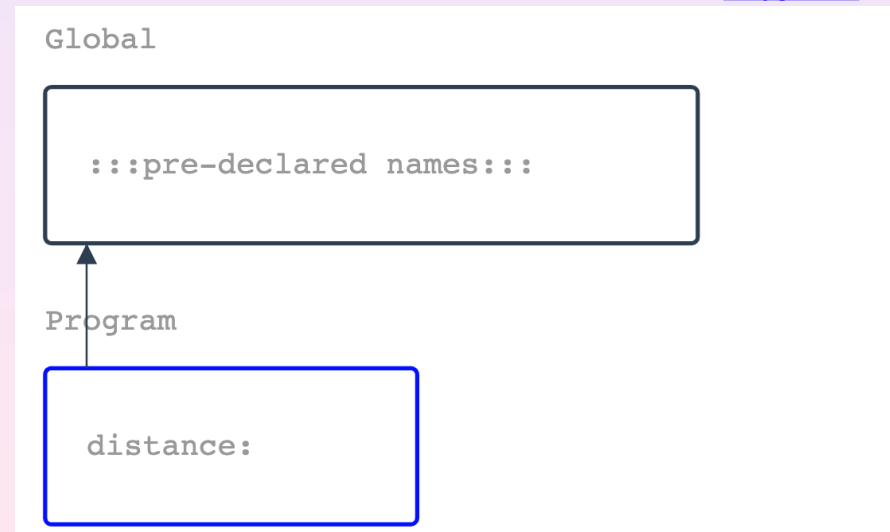
- Need for State; Limitations of Substitution Model
- Introducing State
- Expressions and Declarations
- Control Statements
- Mutable Data

Names refer to locations

Consider

```
let distance = 100;  
distance = 80;  
distance = 50;  
distance = 0;
```

[Show in
Playground](#)

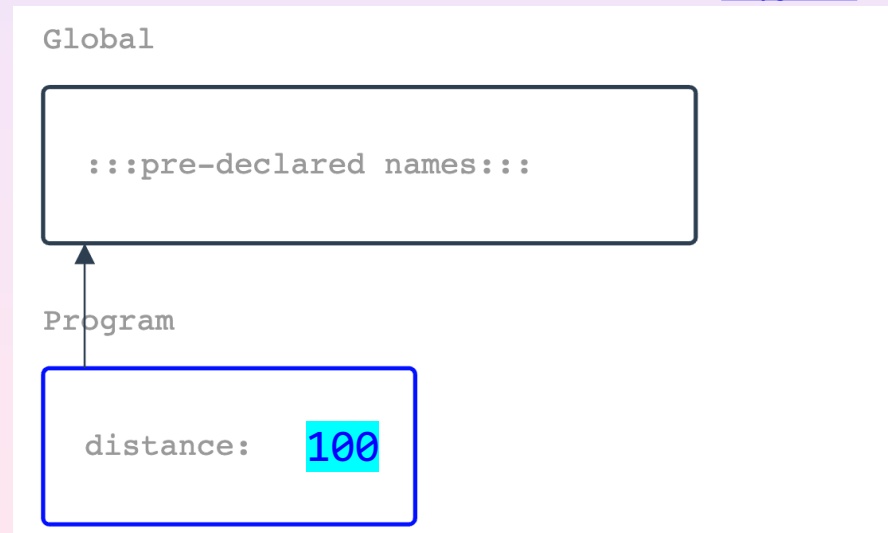


Names refer to locations

Consider

```
let distance = 100;  
distance = 80;  
distance = 50;  
distance = 0;
```

[Show in
Playground](#)

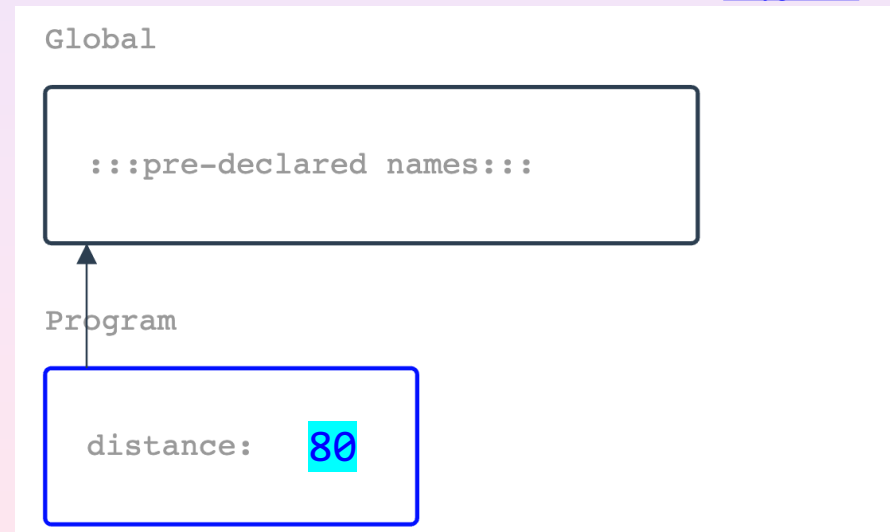


Names refer to locations

Consider

```
let distance = 100;  
distance = 80;  
distance = 50;  
distance = 0;
```

[Show in
Playground](#)

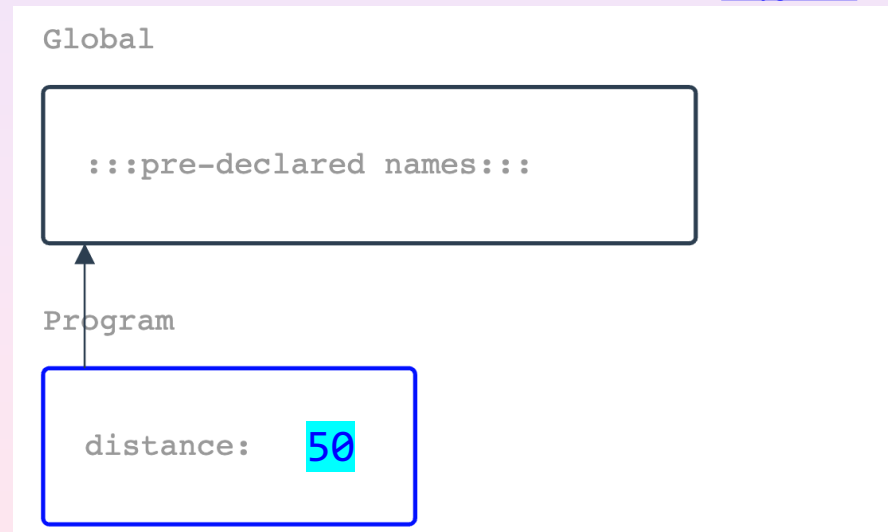


Names refer to locations

Consider

```
let distance = 100;  
distance = 80;  
distance = 50;  
distance = 0;
```

[Show in
Playground](#)

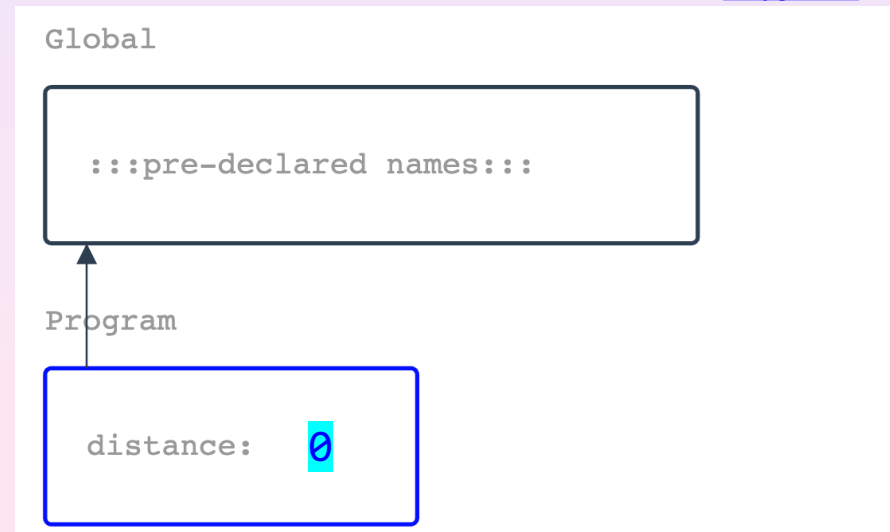


Names refer to locations

Consider

```
let distance = 100;  
distance = 80;  
distance = 50;  
distance = 0;
```

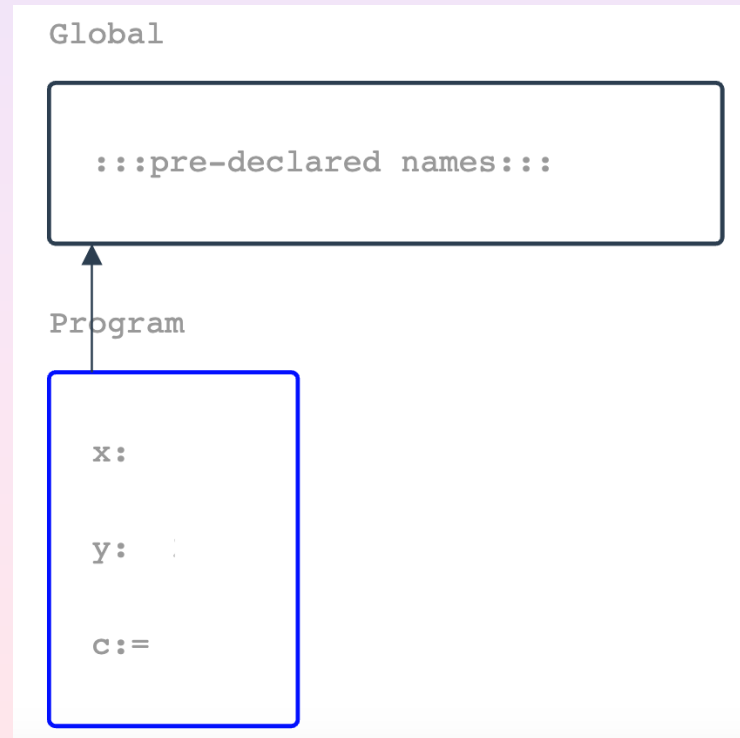
[Show in
Playground](#)



Another example

Consider

```
let x = 10;  
let y = 20;  
const c = 30;  
x = 5;  
y = x;  
y = 3;  
x = c;
```



[Show in
Playground](#)

Declarations and assignments in CSE Machine

- Declarations reserve space in program frame
 - Constant declarations are indicated by `:=`
 - Variable declarations are indicated by `:`
- Execution of declarations and assignments
 - Evaluate their right-hand side expression
 - Change the value in program frame to the result

Outline

- Need for State; Limitations of Substitution Model
- Introducing State
- Expressions and Declarations
- Control Statements
- Mutable Data

Order of Evaluation matters now

Consider

[Show in
Playground](#)

```
let x = 1;  
function f() {  
  x = x + 1;  
  return 0;  
}  
f() + x;
```

CSE machine explicitly prescribes the
order of evaluation of expressions

Expression evaluation: Example 1

Consider

[Show in
Playground](#)

`1 + 2;`

`1 + 2;`

Global

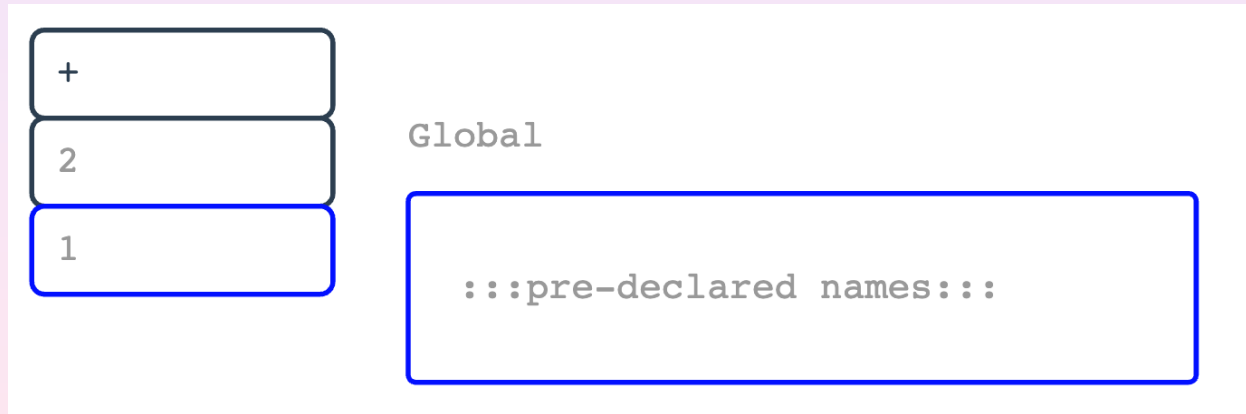
`:::pre-declared names:::`

Expression evaluation: Example 1

Consider

[Show in
Playground](#)

$1 + 2;$

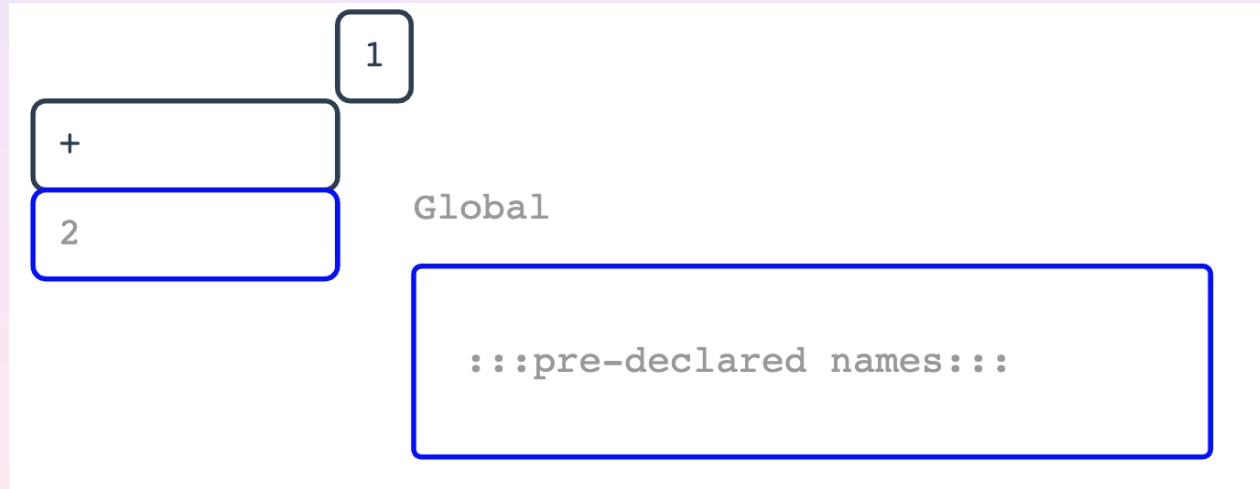


Expression evaluation: Example 1

Consider

$1 + 2;$

[Show in
Playground](#)

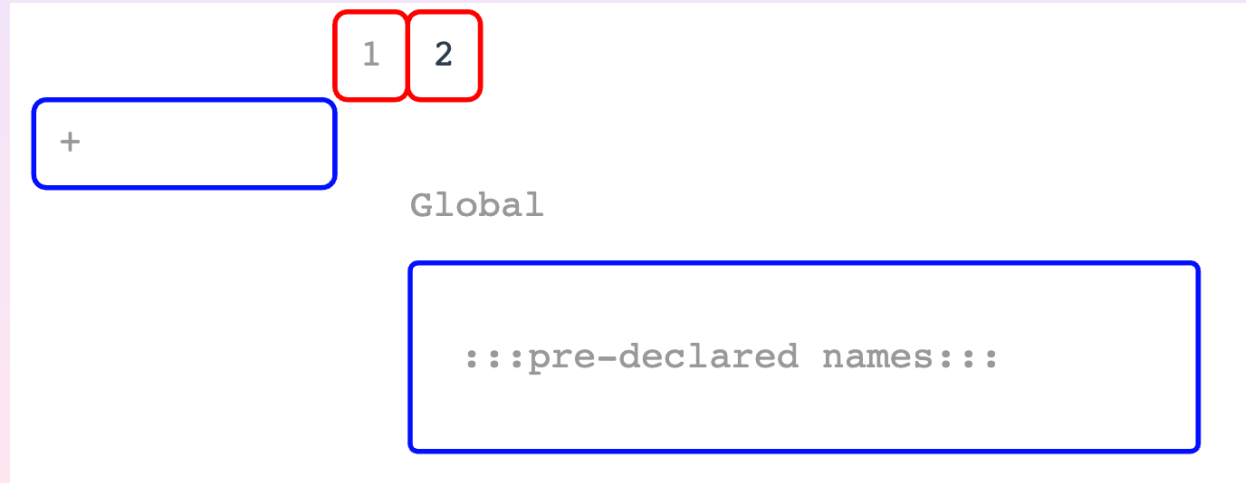


Expression evaluation: Example 1

Consider

[Show in
Playground](#)

$1 + 2;$



Expression evaluation: Example 1

Consider

[Show in
Playground](#)

$1 + 2;$

3

Global

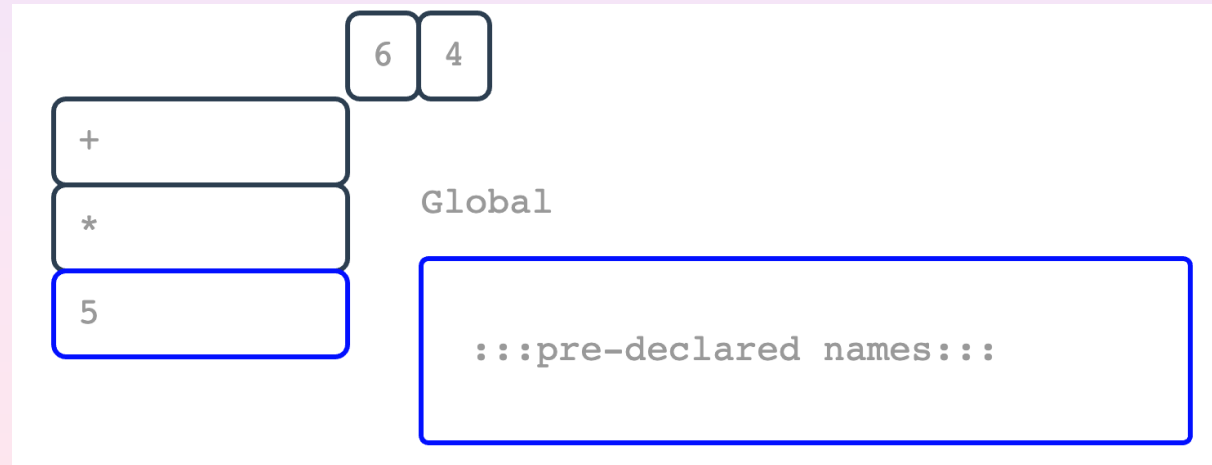
:::pre-declared names:::

Expression evaluation: Example 2

Consider

[Show in
Playground](#)

$2 * 3 + 4 * 5;$

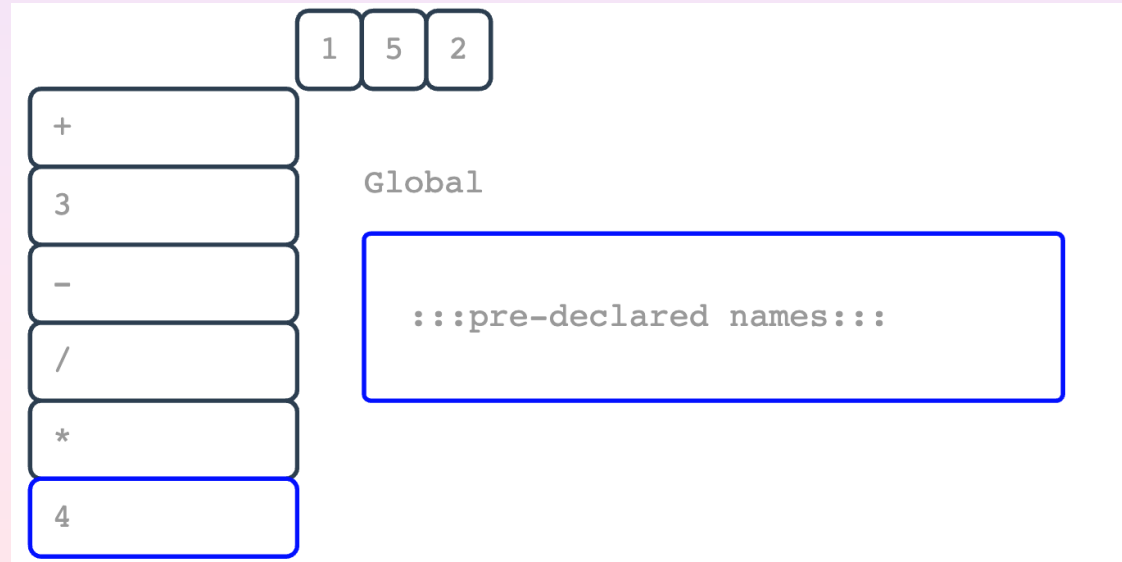


Expression evaluation: Example 3

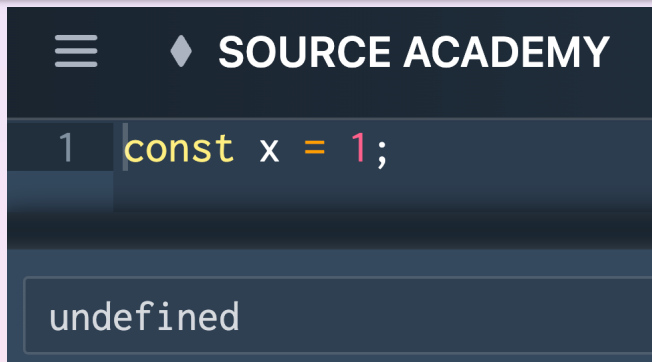
Consider

[Show in
Playground](#)

$1 - 5 / (2 * 4) + 3;$



Why do declarations give undefined?



The screenshot shows a code editor interface for 'SOURCE ACADEMY'. It features a hamburger menu icon and a diamond icon. The code editor displays a single line of code: `1 const x = 1;`. Below the code editor, there is a text box containing the word 'undefined'.

The reason

- Declarations are non-value-producing statements
- If a program does not produce a value, the result is the special value undefined

Constant declaration

- Before program execution: allocate name in frame
- Evaluate right-hand side
- Assign value to name
- POP result from stash

Now

- Stash empty: program produces undefined

Let declarations work exactly like const declarations

The only difference

We prevent assignment to
constants before we even run
the program

Outline

- Need for State; Limitations of Substitution Model
- Introducing State
- Expressions and Declarations
- **Control Statements**
- Mutable Data

Sequences

[Show in
Playground](#)

Consider

```
2 + 3;  
4 / 5;  
6 * 7;
```

What is the significance of “value-producing?”

A sequence produces the value of its **last** value-producing component

Rationale

- Expression statements are value-producing
- pop eliminates their value from stash
- Last result is not popped
- Last result is result of program

Conditional expressions

Consider

[Show in
Playground](#)

```
const x = true;  
x ? 1 + 2 : 3 * 4;
```

Details

- Conditionals need to evaluate predicate and then branch
- Boolean result of predicate will appear on stash
- branch instruction pops Boolean value from stash
- If true, commit to consequent, if false to alternative

Logical composition: &&

[Show in
Playground](#)

Consider

```
const x = true;  
const y = false;  
x && y;
```

Rationale

$E1 \ \&\& \ E2$ is an abbreviation for $E1 \ ? \ E2 \ : \ \text{false}$

Logical composition: &&

[Show in
Playground](#)

Consider

```
const x = true;  
const y = false;  
x || y;
```

Rationale

$E1 \parallel E2$ is an abbreviation for $E1 \text{ ? true : } E2$

Conditional statements

Consider

[Show in
Playground](#)

```
let x = 1;  
if (x > 2) {  
  11;  
} else {  
  22;  
}
```

Rationale

- Conditional statements are treated the same as conditional expressions
- Conditional statements are value-producing

Assignments are value-producing

Consider

[Show in
Playground](#)

```
let x = 0;  
x = x + 1;
```

Details

- asgn instruction leaves value on stash
- Effect: assignment is value-producing

Assignments are value-producing

Consider

[Show in
Playground](#)

```
let x = 0;  
let y = 0;  
let z = 0;  
z = y = x = x + 1;
```

Rationale

- Assignments can be chained together
- All variables are assigned to the same value

What value does a while loop produce?

Consider

[Show in
Playground](#)

```
let i = 0;  
while (i < 1) {  
    i = i + 1;  
}
```

Execution of while loop

- Keep evaluating body as long as predicate evaluates to true
- Keep result of last evaluation as value of while loop

What if the body is not evaluated at all?

Consider

[Show in
Playground](#)

```
let i = 1;  
while (i < 1) {  
  i = i + 1;  
}
```

Execution of while loop

If the body is not evaluated,
its value is undefined

Another example: Factorial

Consider

[Show in
Playground](#)

```
let n = 5;  
let i = 1;  
let result = 1;  
while (i < n) {  
    i = i + 1;  
    result = result * i;  
}
```

How are while loops evaluated in CSE machine?

Consider

[Show in
Playground](#)

```
let i = 1;  
while (i < 1) {  
    i = i + 1;  
}
```

Details

- While loop pushes undefined, predicate and while instruction on control
- while instruction: if predicate value is false: done
- while instruction: if predicate value is true:
pop, body, predicate, same while instruction

For loops: Factorial example

Consider

[Show in
Playground](#)

```
const n = 5;  
let i = undefined;  
let result = 1;  
for (i = 1; i <= n; i = i + 1) {  
    result = result * i;  
}
```

Rationale

- Evaluate initialization instruction
- Keep evaluating body as long as predicate evaluates to true
- After each body evaluation, evaluate increment expression

For loops: Factorial example

Consider

[Show in
Playground](#)

```
const n = 5;  
let i = undefined;  
let result = 1;  
for (i = 1; i <= n; i = i + 1) {  
    result = result * i;  
}
```

Details

- For loop leads to undefined, init, pop, predicate, for instruction
- for instruction pops predicate value from stash
- If value is false: finished
- If value is true: pop, body, increment, pop, predicate, for instr

Outline

- Need for State; Limitations of Substitution Model
- Introducing State
- Expressions and Declarations
- Control Statements
- Mutable Data

Mutable Data

- **Assignment** gives us the ability to create *mutable* data, i.e. data that can be modified
 - E.g. result in factorial while loop
- In Source §1 and §2, all our data were *immutable*. We had
 - Constructors, selectors, predicates, printers
 - But no *mutators*

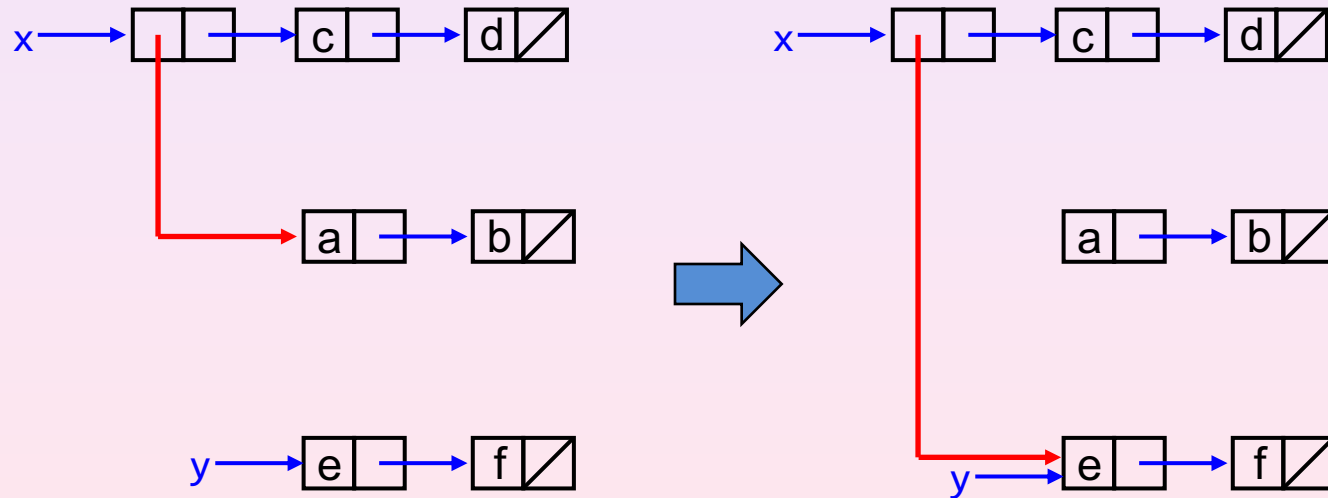
Mutable Pairs

- Now we will allow **mutators** in order to create **mutable data structures**
- After creating a pair with `pair`
 - The **head** can be changed using `set_head`
 - The **tail** can be changed using `set_tail`
- Mutating mutable pairs
 - `set_head(p, x)` changes **head** of pair `p` to `x`
 - `set_tail(p, x)` changes **tail** of pair `p` to `x`

[Show in
Playground](#)

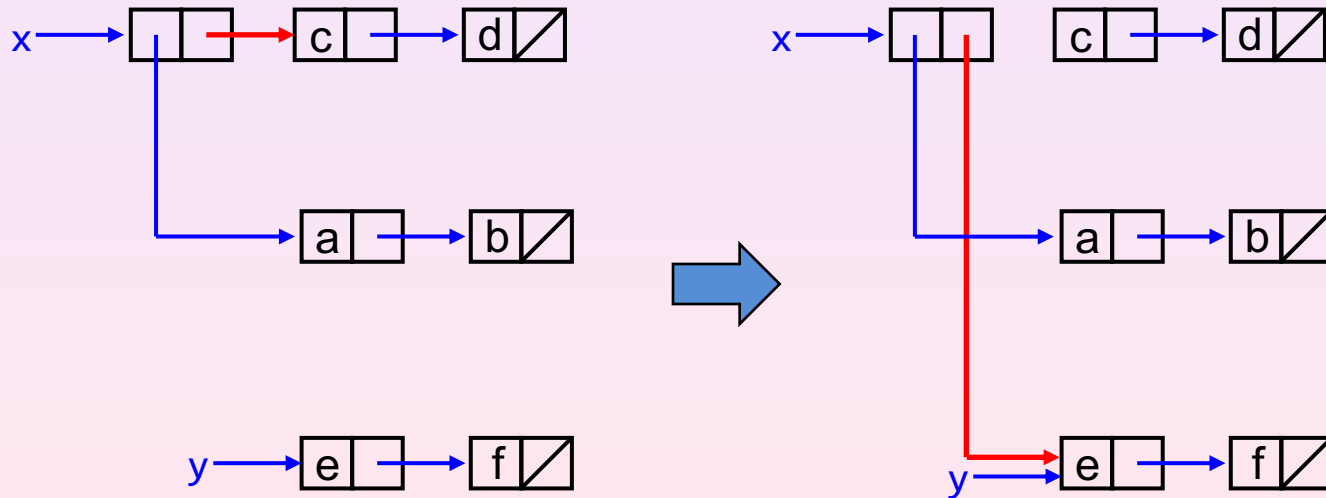
set_head Example

- Effect of `set_head(x, y)`



set_tail Example

- Effect of `set_tail(x, y)`



Be Careful with Mutators!

- **Example:**

```
const a = list(1, 3, 5);  
const b = list(2, 4);  
const c = append(a, b);  
c; → [1, [3, [5, [2, [4, null]]]]]
```

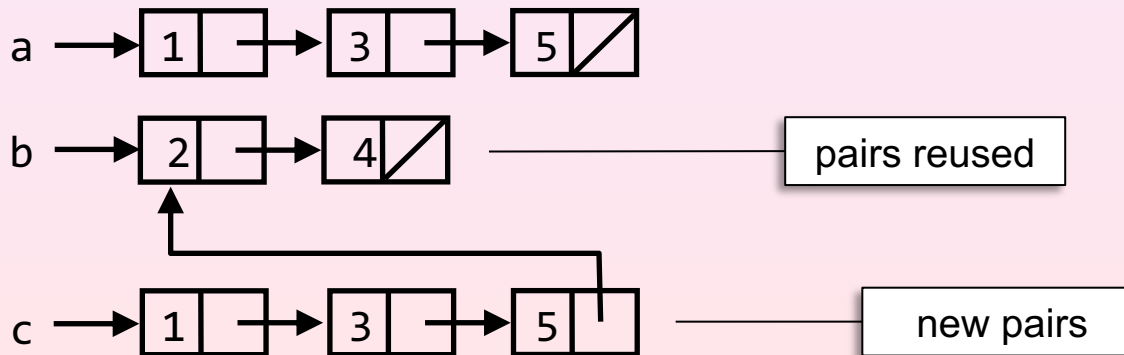
```
set_head(b, 9);  
b; → [9, [4, null]]  
c; → [1, [3, [5, [9, [4, null]]]]]
```

- Mutating **b** changes **c** as well !!!
- What is happening?

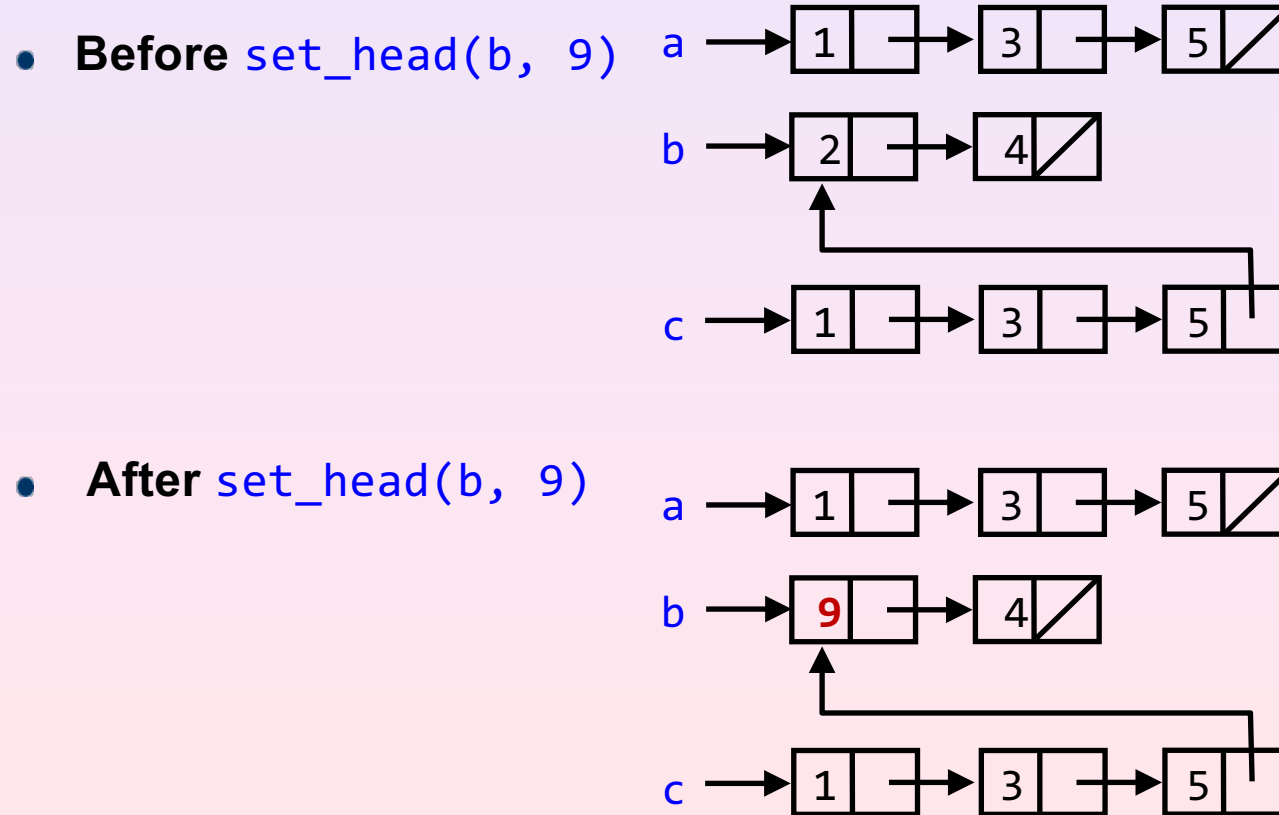
Recall the append function

```
function append(xs, ys) {  
  return is_null(xs)  
    ? ys  
    : pair(head(xs), append(tail(xs), ys));  
}  
const a = list(1, 3, 5);  
const b = list(2, 4);  
const c = append(a, b);
```

[Show in
Playground](#)



Mutation and Sharing

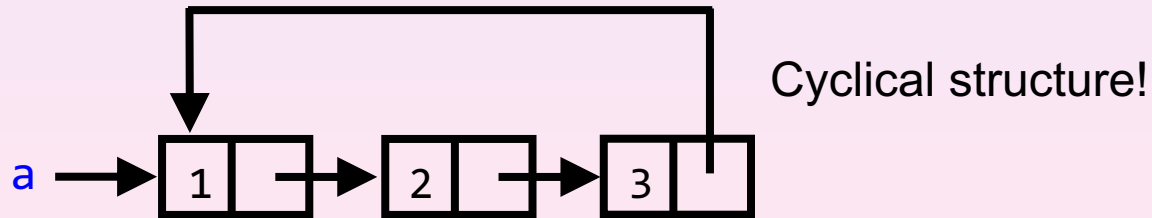


Be Careful with Mutators!

[Show in
Playground](#)

- Another example:

```
const a = list(1, 2, 3);  
set_tail(tail(tail(a)), a);
```



- What is `length(a)`?!

Mutable (“Destructive”) List Processing — Append

- **Wanted:**

A function to **append** two lists and return the result list

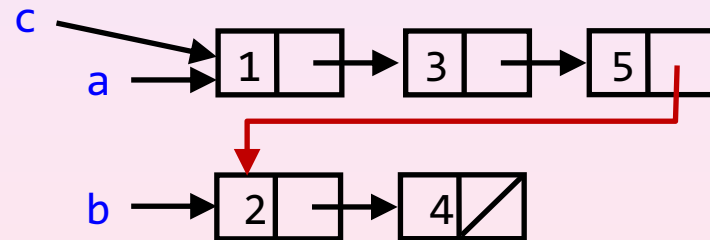
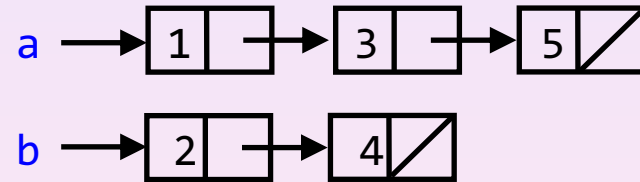
- **No new pair must be created**
- Result list is constructed from existing pairs of input lists

“Destructive” Append

- Example:

```
const a = list(1, 3, 5);  
const b = list(2, 4);
```

```
const c = d_append(a, b);
```



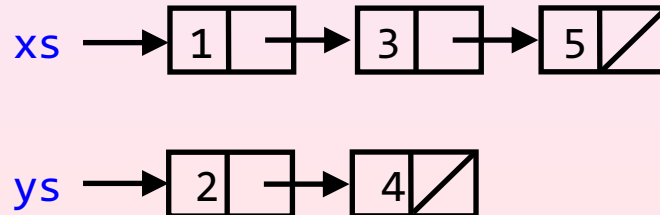
```
c; → [1, [3, [5, [2, [4, null]]]]]  
a; → [1, [3, [5, [2, [4, null]]]]]  
b; → [2, [4, null]]
```

“Destructive” Append

- Implementation:

```
function d_append(xs, ys) {  
  if (is_null(xs)) {  
    return ys;  
  } else {  
    set_tail(xs, d_append(tail(xs), ys));  
    return xs;  
  }  
}
```

[Show in
Playground](#)



“Destructive” Map

- Example:

`const a = list(1, 2, 3);` $a \rightarrow \boxed{1 \mid -} \rightarrow \boxed{2 \mid -} \rightarrow \boxed{3 \mid /}$

$a; \rightarrow [2, [4, [6, []]]]$

`d_map(x => x * 2, a);` $a \rightarrow \boxed{2 \mid -} \rightarrow \boxed{4 \mid -} \rightarrow \boxed{6 \mid /}$

$a; \rightarrow [2, [4, [6, []]]]$

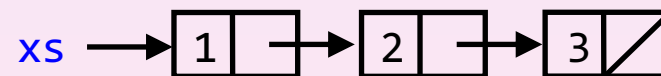
“Destructive” Map

- Implementation:

```
function d_map(fun, xs) {  
  if (!is_null(xs)) {  
    set_head(xs, fun(head(xs)));  
    d_map(fun, tail(xs));  
  } else { }  
}
```

[Show in
Playground](#)

```
const xs = list(1, 2, 3);  
d_map(x => x + 1, xs);
```

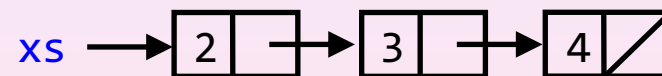


“Destructive” Map

- Implementation:

```
function d_map(fun, xs) {  
  if (!is_null(xs)) {  
    set_head(xs, fun(head(xs)));  
    d_map(fun, tail(xs));  
  } else { }  
}
```

[Show in
Playground](#)



```
const xs = list(1, 2, 3);  
d_map(x => x + 1, xs);
```

Summary

- **Variables** and **assignment** allow us to create **state**
- **While** loops and **for** loops let us program **iterative** processes
- **Substitution model** breaks down with **assignment**
- **CSE machine** lets us see the execution of programs
- `set_head` and `set_tail` mutate **pairs** & **lists**
- Be careful when mutating things!